ReactJS

Unit 3 — State & Events

Hooks · useState · Events · Forms · Controlled components . Conditional Rendering

In This Unit

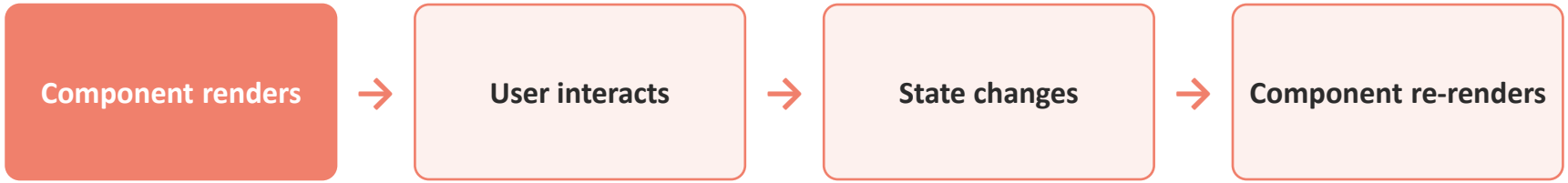
1. What is state?
2. Hooks and useState
3. Reading and updating state
4. React events
5. Form basics
6. Controlled components
7. Conditional Rendering

STATE & HOOKS

Data that changes · Components re-render

What is State?

- State = data inside a component that can change
- When state changes, React re-renders the component
- The component displays the new data



What is a Hook?

- A Hook is a React function that lets you use features
- useState is a Hook that tracks state in function components
- Hooks start with 'use': useState, useEffect, useContext

Always import useState

```
import { useState } from 'react';
```

useState: The Basics

- useState returns two things:
- 1. The current value of state
- 2. A function to update that value

```
const [count, setCount] = useState(0);
// count = the current value
// setCount = the function to update it
// 0 = the initial value
```

Reading State

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return <h1>Count: {count}</h1>;
}
```

Result

Count: 0

Updating State

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        +
      </button>
    </div>
  );
}
```

What State Can Hold

- String: const [name, setName] = useState('Ravi');
- Number: const [age, setAge] = useState(20);
- Boolean: const [isActive, setIsActive] = useState(true);
- Object: const [user, setUser] = useState({name: '', age: 0});
- Array: const [items, setItems] = useState([]);

Multiple State Variables

```
function User() {
  const [name, setName] = useState('');
  const [age, setAge] = useState(0);
  const [email, setEmail] = useState('');

  return (
    <div>
      <p>Name: {name}</p>
      <p>Age: {age}</p>
      <p>Email: {email}</p>
    </div>
  );
}
```

EVENTS

User interactions · Click, Change, Submit

React Events: camelCase

HTML	React
onclick	onClick
onchange	onChange
onsubmit	onSubmit
onmouseover	onMouseOver

React syntax (not onclick=)

```
<button onClick={handleClick}>
  Click me
</button>
```

onClick: Simple Handler

```
function Button() {
  function handleClick() {
    alert('Button clicked!');
  }

  return (
    <button onClick={handleClick}>
      Click
    </button>
  );
}
```

- Pass the function name (no parentheses)
- Clicking runs handleClick()

onClick with Arguments

Arrow function for arguments

```
<button onClick={() =>
  setCount(count + 1)
}
>
  +
</button>
```

Access the event

```
<button onClick={(e) =>
  console.log(e)
}
>
  Click
</button>
```

Use () => {} when you need to pass data or use the event.

onChange: Text Input

```
function Input() {
  const [text, setText] = useState('');

  return (
    <div>
      <input
        type="text"
        value={text}
        onChange={(e) => setText(e.target.value)}
      />
      <p>You typed: {text}</p>
    </div>
  );
}
```

onChange: Select Dropdown

```
function Select() {
  const [color, setColor] = useState('red');

  return (
    <div>
      <select value={color}
        onChange={(e) => setColor(e.target.value)}>
        <option>red</option>
        <option>blue</option>
        <option>green</option>
      </select>
      <p>Color: {color}</p>
    </div>
  );
}
```

FORMS

Controlled components · Submit

Controlled Components

- Controlled = React state is the source of truth
- Input value comes from state
- onChange updates state
- Usually for <input>, <textarea>, <select>

```
<input value={name} onChange={(e) => setName(e.target.value)} />
```

Form with Submission

```
function Form() {
  const [name, setName] = useState('');

  function handleSubmit(e) {
    e.preventDefault();
    alert('Hello ' + name);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={name}
        onChange={(e) => setName(e.target.value)} />
      <button type="submit">Submit</button>
    </form>
  );
}
```

e.preventDefault(): What Does It Do?

- Default behavior of a form: page reloads when submitted
- preventDefault() stops this default behavior
- Lets React handle the form instead

```
function handleSubmit(e) {
  e.preventDefault();
  // No page reload
}
```

Multiple Inputs with One State

```
const [form, setForm] = useState(
  {name: '', email: ''}
);

function handleChange(e) {
  setForm({...form,
    [e.target.name]: e.target.value
  });
}

<input name="name" value={form.name}
       onChange={handleChange} />
<input name="email" value={form.email}
       onChange={handleChange} />
```

CONDITIONAL RENDERING

if/else · Ternary · Show/hide

What is Conditional Rendering?

- Conditional rendering = showing or hiding content based on a condition
- Like an if/else statement in JavaScript
- Common in React: logged in? show Dashboard, else show Login

```
if (isLoggedIn) {
  show Dashboard
} else {
  show Login
}
```

Method 1: Ternary Operator (?:)

```
const [loggedIn, setLoggedIn] = useState(false);

<div>
  {loggedIn ? (
    <h1>Welcome back!</h1>
  ) : (
    <h1>Please log in</h1>
  )}
</div>
```

- condition ? (if true) : (if false)
- Most common pattern in React
- Can inline in JSX

Method 2: Logical AND (&&)

```
const [showMessage, setShowMessage] =
  useState(true);

<div>
  {showMessage && (
    <p>This only shows if true</p>
  )}
</div>
```

- True && show content
- False && show nothing
- Good for simple show/hide

Method 3: if/else Before Return

```
function Login() {
  const [loggedIn, setLoggedIn] =
    useState(false);

  if (loggedIn) {
    return <h1>Welcome!</h1>;
  }

  return <h1>Log in first</h1>;
}
```

- Use if/else before the return statement
- Also called early return
- Good for complex logic

Example: Toggle Button

```
function Toggle() {
  const [show, setShow] = useState(false);

  return (
    <div>
      <button onClick={() => setShow(!show)}>
        {show ? 'Hide' : 'Show'}
      </button>
      {show && <p>You can see me!</p>}
    </div>
  );
}
```

Conditional CSS Classes

```
const [active, setActive] = useState(false);

<div className={active ? 'active' : 'inactive'}>
  Content
</div>

{/* or using && */}
<div className={'btn ' + (active && 'btn-active')}>
  Content
</div>
```

Practice

- Create a Counter with +/- buttons
- Make a TextInput that shows what you type
- Build a Form with name and email fields
- Use preventDefault() to stop page reload
- Track state with useState

Summary

- State = component data that changes
- useState = Hook to track state in function components
- Events are camelCase: onClick, onChange, onSubmit
- Controlled components use React state for input values
- Call e.preventDefault() in form submit handlers